

ZUDOBOT v1 — COMPREHENSIVE BUSINESS REQUIREMENTS DOCUMENT

Document ID: DVS-ZUDOBOT-SPEC-20260424-V2.0.1
Date: April 24, 2025
Version: 2.0.1
Status:  Approved — Under Development (Sprint 1)
Platform: Dives Space (Main Domain + Sub-Domain)
AI Engine: Google Gemini 2.0 Flash + Gemini 1.5 Pro
Target Market: E-commerce merchants, SaaS multi-tenant platform
Development Repository: <https://github.com/din-zudogu/zudobot>
Production Deployment: AWS Amplify (zudobot)
Local Development Path: C:\zudobot-saas

TABLE OF CONTENTS

- 1. [Executive Summary](#)
- 2. [Vision & Business Objectives](#)
- 3. [Stakeholders & User Personas](#)
- 4. [System Architecture Overview](#)
- 5. [AI Engine Specification](#)
- 6. [Database Schema & Data Models](#)
- 7. [Constitutional Rules & AI Safety](#)
- 8. [Service Architecture & Business Logic](#)
- 9. [User Journeys & Workflows](#)
- 10. [Feature Complete Map](#)
- 11. [Product Pricing & Monetization](#)
- 12. [Billing System & Usage Tracking](#)
- 13. [LINE Notify Integration](#)
- 14. [Development Phases & Timeline](#)
- 15. [Success Metrics & KPIs](#)
- 16. [Risks & Mitigation Strategies](#)
- 17. [PDPA Compliance & Data Privacy](#)
- 18. [Future Roadmap](#)

1. EXECUTIVE SUMMARY

What is Zudobot?

Zudobot v1 is an AI-powered Sales Agent Chatbot designed to serve as a 24/7 sales employee for e-commerce merchants on the **Dives Space** platform. Unlike traditional FAQ bots that passively answer questions, Zudobot actively:

- **Proactively engages** customers based on behavioral triggers
- **Remembers customer preferences** across multiple sessions
- **Closes sales** by generating personalized checkout links
- **Handles objections** intelligently using pre-configured sales strategies
- **Tracks customer sentiment** in real-time
- **Hands off to human staff** seamlessly when needed
- **Operates 24/7** without fatigue or errors

The product is **underpinned by Google Gemini**, protected by immutable **Constitutional Rules** (กฎเหล็ก) that ensure ethical behavior, and designed for **multi-tenant SaaS** architecture with strict tenant isolation.

Core Value Proposition

Aspect	Traditional Bot	Zudobot v1
Capability	Answers FAQs	Persuades & Closes Sales
Engagement	Passive (waits for customer)	Proactive (initiates conversations)
Memory	Forgets after session	Remembers across sessions
Context	Generic responses	Personalizes to each customer
Tone	Generic/Script	Mirrors customer personality
Integration	None	LINE Notify alerts per-tenant
Operability	Manual setup	AI Auto-setup
Sales Closing	N/A	Direct checkout link in chat
Sentiment	N/A	Real-time sentiment scoring
Human Escalation	Manual	Smart triggers + Human handoff

Business Impact (Target KPIs)

- **+25% Conversion Rate** (short term, Dives Space merchants)
- **+15% Average Order Value** (via Upsell/Cross-sell)
- **70% reduction** in merchant's chat response burden
- **Recurring Revenue** from AI Package Subscription

- **White-label expansion** to external SME market by Phase 2

2. VISION & BUSINESS OBJECTIVES

2.1 Vision Statement (วิสัยทัศน์)

"Every merchant on Dives Space has access to a world-class AI sales agent that works 24 hours a day, never takes a break, never gets sick, and increases revenue every single day."

2.2 Mission Statement (พันธกิจ)

To democratize AI-powered sales conversations by providing affordable, easy-to-use, culturally-aware chatbot technology that helps merchants close more sales without hiring expensive sales staff.

2.3 Short-term Objectives (3–6 months, within Dives Space)

1. **Increase Conversion Rate:** +25% for participating merchants
2. **Increase AOV:** +15% through smart Upsell/Cross-sell recommendations
3. **Reduce Merchant Burden:** 70% fewer manual chat responses needed
4. **Generate Recurring Revenue:** Establish SaaS pricing model for Zudobot packages
5. **Pilot with 3–5 Merchants:** Validate product-market fit within Dives Space
6. **Build Trust:** Zero safety violations, zero constitutional rule breaches

2.4 Medium-term Objectives (6–12 months, External Market)

1. **White-label Launch:** Zudobot available as standalone SaaS product
2. **SME Market Entry:** Target small businesses (50–500 employees) that can't afford custom bot development
3. **Widget Integration:** 1-line JavaScript embed on any website
4. **Multi-channel Support:** LINE OA integration by Phase 2
5. **Third-party Platform Support:** Shopify, WooCommerce connectors
6. **Ecosystem Revenue:** 40% of new revenue from outside Dives Space

2.5 Long-term Vision (12+ months)

- **Multi-language Support:** Thai, English, Vietnamese, Indonesian
- **Multi-channel Hub:** LINE OA, Facebook Messenger, Instagram DM, WhatsApp, SMS
- **Advanced Analytics:** CRM auto-tagging, knowledge gap detection, predictive churn
- **Industry Verticals:** Pre-built templates for Fashion, F&B, Beauty, Real Estate
- **Competitive Advantage:** Proprietary customer memory + constitutional rules become IP moat

3. STAKEHOLDERS & USER PERSONAS

3.1 Primary Stakeholders

Stakeholder	Role	Interest	Pain Point
End-User (Shopper)	Browses & buys	Quick answers, personalized experience	Long wait times, generic bots
Merchant/Shop Owner	Configures & monitors bot	Increase sales, reduce workload	Can't afford 24/7 staff
Dives Space Admin	Platform owner & moderator	New revenue stream, platform stickiness	Quality control, billing complexity
Dives Space End-User (Merchant)	Uses on their store	ROI on bot service	Setup complexity, unclear value
External SaaS Customer (Phase 2)	White-label end-user	Customizable bot for their website	Technical complexity

3.2 User Personas

Persona 1: Nee — Busy Shop Owner (Primary)

- **Age:** 35–50
- **Business:** Fashion E-commerce store (Dives Space member)
- **Tech Savvy:** Low–Medium
- **Annual Revenue:** \$2–10M
- **Pain:** Can't hire 24/7 chat staff, loses sales after-hours
- **Motivation:** "I want the bot to sell for me while I sleep"
- **Success Metric:** +20% revenue from night-time chats, less tired from answering questions

Persona 2: Alex — Tech-Savvy Developer (Secondary)

- **Age:** 25–35
- **Role:** Technical co-founder or CTO of SME
- **Tech Savvy:** High
- **Annual Revenue:** \$5–50M
- **Pain:** Don't want to pay \$500K for custom bot dev, want quick deploy
- **Motivation:** "Give me a widget I can drop on my site in 5 minutes"
- **Success Metric:** Installed on 5+ sites within month, zero technical support needed

Persona 3: Din — Platform Admin (Tertiary)

- **Age:** 30–45
- **Role:** Dives Space Platform Owner/Manager

- **Tech Savvy:** High
- **Pain:** New revenue stream needed, competitors have AI features
- **Motivation:** "I need to offer this to keep merchants happy and generate recurring MRR"
- **Success Metric:** \$50K+ MRR from Zudobot subscriptions by end of year

Persona 4: Pim — CRM Manager (Tertiary)

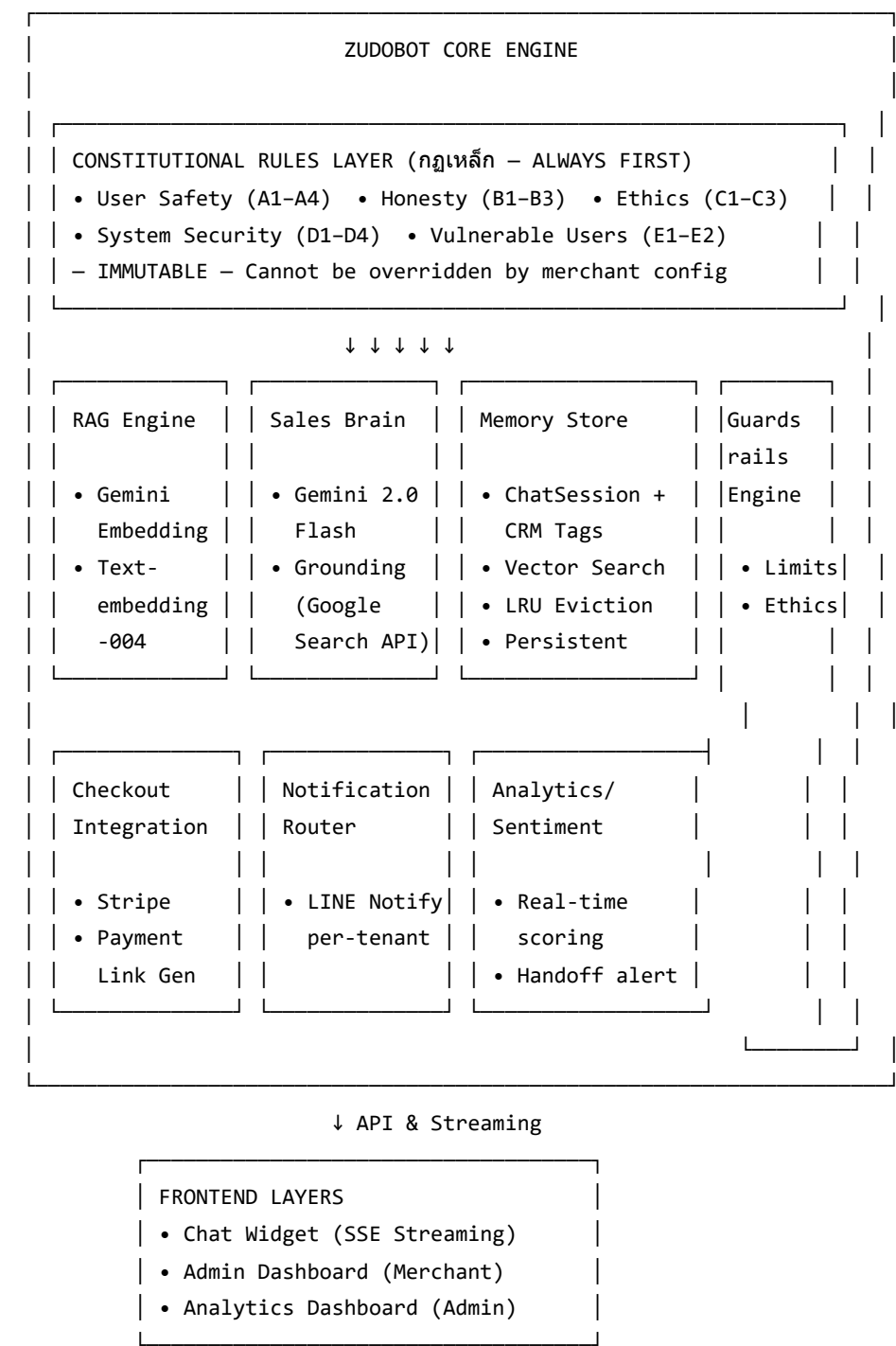
- **Age:** 28–40
- **Role:** Marketing/Analytics Manager in medium business
- **Tech Savvy:** Medium
- **Pain:** Need to understand which chats convert, who's a VIP, what questions repeat
- **Motivation:** "Show me the analytics, let me optimize"
- **Success Metric:** Dashboard shows which products are most asked about, which visitors convert

3.3 Stakeholder Engagement Plan

Phase	Merchant	Admin	End-User	Developer
Phase 1 (MVP)	3 pilot merchants	Monthly reviews	Surveys weekly	—
Phase 2	Expand to 20 merchants	Weekly syncs	Beta UAT	Early access
Phase 3	Full rollout	Operational support	Public launch	API docs
Phase 4	White-label partners	Revenue sharing	Multi-platform	Partner program

4. SYSTEM ARCHITECTURE OVERVIEW

4.1 High-Level Architecture Diagram

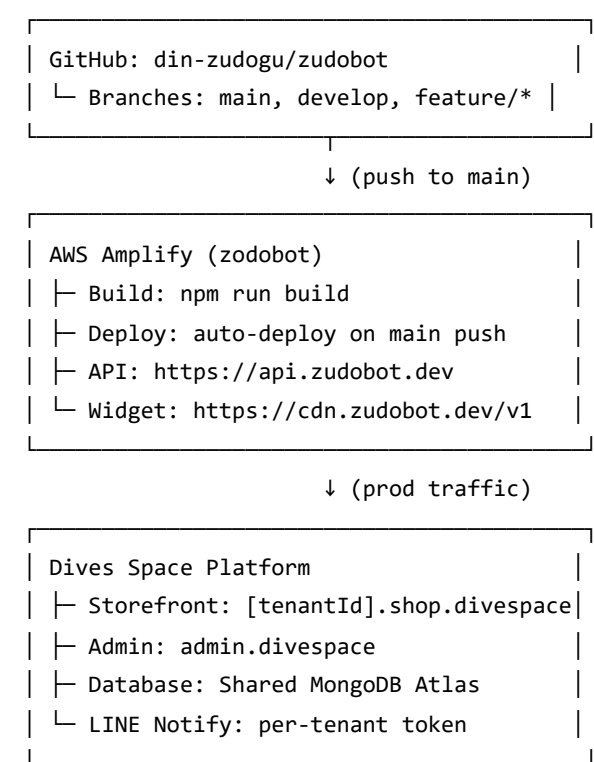


4.2 Technology Stack

Layer	Technology	Purpose
AI Engine	Google Gemini 2.0 Flash	Real-time chat responses

Layer	Technology	Purpose
AI Engine	Google Gemini 1.5 Pro	Complex reasoning, sentiment analysis
Embedding	text-embedding-004	Vector RAG for product/knowledge retrieval
Vector DB	MongoDB Atlas Vector Search	Semantic similarity search
Cache/Session	MongoDB ChatSession	Conversation persistence (TTL: 30 days)
Customer Memory	VisitorMemoryEntry + Vector Index	Cross-session context injection
Backend	Next.js App Router (TypeScript)	API routes + Server-Side Rendering
Database	MongoDB + Atlas	Document storage + vector indexing
Real-time Chat	Server-Sent Events (SSE)	Streaming bot responses
Notifications	LINE Notify API	Per-tenant alerts to merchant
Payment	Stripe API	Embedded payment link generation
Auth	API Key (x-api-key, x-secret-key)	Server-to-server + widget integration
Hosting	AWS Amplify	Production deployment (zodobot)
Version Control	GitHub	https://github.com/din-zudogu/zodobot
laC	amplify.yml	Infrastructure as Code

4.3 Deployment Architecture



4.4 Tenant Isolation Strategy

Principle: Multi-tenant architecture with strict data compartmentalization.

1. **Data Isolation**

- Every query filters by `tenantId` (indexed)
- ChatSession namespace per tenant
- Knowledge Base per tenant
- Products per tenant
- Visitor Memory per tenant

2. **Token Isolation**

- LINE Notify Token stored in `tenant.aiAgent.lineNotifyToken`
- Each tenant gets distinct, non-transferable token
- Merchant A's alerts only go to Merchant A's LINE

3. **Config Isolation**

- BotConfig unique per tenant
- Custom persona per tenant
- Guardrails per tenant
- Operating hours per tenant

4. **API Key Isolation**

- `publicKey` (widget auth): tenant-specific, checked via `allowedDomains`
- `secretKey` (server-to-server): tenant-specific
- All incoming requests validated for tenant match

5. AI ENGINE SPECIFICATION

5.1 Why Google Gemini (vs OpenAI)?

Criterion	Gemini	OpenAI
Thai Language Support	Excellent (trained on large Thai dataset)	Good
Grounding (Real-time Search)	✔ Native Google Search integration	✗ Requires manual implementation
Context Window	1M tokens (Gemini 1.5 Pro)	128K tokens (GPT-4)
Cost per 1K Tokens	\$0.000075 (2.0 Flash)	\$0.03 (4o)
Latency	~0.5s (Flash)	~1–2s (4o)
Multimodal	✔ Native (image + video)	✔ Limited
Ecosystem	Google Workspace, Ads, Analytics	Closed ecosystem

Verdict: Gemini is ideal for high-volume, cost-sensitive, real-time conversational AI with Thai language nuance.

5.2 Model Selection by Use Case

Use Case	Model	Reasoning
Real-time Chat (PRIMARY)	Gemini 2.0 Flash	Fastest, cheapest, 95% quality for sales conversations
Complex Reasoning	Gemini 1.5 Pro	Used for sentiment analysis, objection detection, importance scoring
Embeddings	text-embedding-004	768-dim vectors for product RAG + customer memory
LINE Auto-setup	Gemini 1.5 Flash	Generate merchant-specific handoff messages & bot intro

5.3 API Integration Details

Base URL: `https://generativelanguage.googleapis.com/v1beta/models`

Key Endpoints:

- `POST /gemini-2.0-flash:streamGenerateContent` — Streaming chat
- `POST /gemini-1.5-pro:generateContent` — Batch reasoning
- `POST /text-embedding-004:embedContent` — Vector generation

Authentication: Bearer token via `GEMINI_API_KEY` environment variable

Rate Limits:

- Free tier: 15 requests/min, 500K free tokens/month
- Paid tier: Scalable, ~\$0.075 per 1M tokens (2.0 Flash)

Error Handling:

- 429 Too Many Requests → Exponential backoff (1s → 2s → 4s)
- 500 Service Unavailable → Fallback to graceful error message
- Safety filter block → Log to RuleViolation, send apology message

5.4 Gemini Grounding (Real-time Research)

What: Gemini can search Google in real-time for current information.

When to Use: Bot doesn't find answer in Knowledge Base → fallback to Google Search → cite source

Flow:

1. User asks: "สกุลเงินไทยวันนี้เท่าไร?"
2. Bot's Knowledge Base is empty

3. Gemini performs Google Search Grounding
4. Returns: "ตามข้อมูลล่าสุดจาก Google..."
5. Cites source with attribution

Risk Mitigation: Only use for non-sales content (product prices always from KnowledgeBase only)

5.5 Safety Settings

Gemini Built-in Safety Filters:

```
const SAFETY_SETTINGS = [  
  { category: HARM_CATEGORY_HARASSMENT, threshold: BLOCK_MEDIUM_AND_ABOVE },  
  { category: HARM_CATEGORY_HATE_SPEECH, threshold: BLOCK_MEDIUM_AND_ABOVE },  
  { category: HARM_CATEGORY_SEXUALLY_EXPLICIT, threshold: BLOCK_MEDIUM_AND_ABOVE },  
  { category: HARM_CATEGORY_DANGEROUS_CONTENT, threshold: BLOCK_MEDIUM_AND_ABOVE },  
];
```

These settings ensure that even if a customer tries to get the bot to say harmful content, Gemini refuses at the API layer.

6. DATABASE SCHEMA & DATA MODELS

6.1 Core Models Overview

The Zudobot system uses **13 primary data models**, all stored in **MongoDB**.

Model 1: Tenant — SaaS Account (Multi-tenant Root)

Purpose: Represents one shop/merchant on Dives Space or external white-label customer.

```
{  
  _id: ObjectId,  
  name: String,                // e.g., "บ้านตัดผม นายวิชัย"  
  publicKey: String,           // Widget auth (x-api-key), unique  
  secretKey: String,           // Server-to-server auth, unique  
  allowedDomains: String[],    // Domain whitelist for CORS  
  plan: "trial" | "starter" | "pro" | "enterprise",  
  expiryDate: Date | null,     // Subscription expiry  
  isActive: Boolean,           // Can use bot or not  
  lineNotifyToken: String,     // Per-tenant LINE token (V2.0.1)  
  lineNotifyEnabled: Boolean,   // Toggle LINE alerts  
  createdAt: Date,  
  updatedAt: Date,  
}
```

Indexes: publicKey (unique), secretKey (unique), plan , isActive

Model 2: PackageConfig — Pricing & Quota Master Data

Purpose: Platform-wide pricing catalog. Merchants don't modify this; Platform Admin does.

```
{
  _id: ObjectId,
  slug: String,                // "trial", "starter_monthly", "addon_msg_1000", ...
  packageType: "BASE_PLAN" | "ADDON_MESSAGES" | "ADDON_MEMORY",
  name: String,                // "Zudobot Starter", "Extra 1K Messages", ...
  description: String,         // Marketing description
  price: Number,               // THB (0 = free)
  messageQuota: Number,        // Monthly messages allowed (0 = N/A)
  visitorMemoryQuota: Number,  // # of visitors can be remembered (0 = N/A)
  billingCycle: "monthly" | "one_time",
  isActive: Boolean,           // Available for purchase or not
  sortOrder: Number,          // Display order in dashboard
  createdAt: Date,
  updatedAt: Date,
}
```

Example Records:

- { slug: "trial", name: "Free Trial", price: 0, messageQuota: 100, visitorMemoryQuota: 0 }
- { slug: "starter_monthly", name: "Starter", price: 490, messageQuota: 2000, visitorMemoryQuota: 500 }
- { slug: "addon_msg_1000", name: "Extra 1K Messages", price: 99, messageQuota: 1000, visitorMemoryQuota: 0 }

Indexes: slug (unique), packageType , isActive

Model 3: TenantUsage — Real-time Usage Tracking (ONE per Tenant)

Purpose: Tracks current consumption vs. quota for each tenant, including billing cycle management.

```

{
  _id: ObjectId,
  tenantId: ObjectId,           // Foreign key to Tenant (unique)
  activePackageSlug: String,    // Currently active base plan
  addons: [
    {
      packageId: ObjectId,      // FK to PackageConfig
      packageSlug: String,      // Denormalized for readability
      purchasedAt: Date,
      expiresAt: Date | null,    // null = no expiry
      quotaGranted: Number,      // Messages/visitors added by addon
    }
  ],
  // Message quota
  totalMessageQuota: Number,    // Base + sum(active addon messages)
  usedMessages: Number,        // +1 every time bot sends a message
  // Visitor memory quota
  totalVisitorMemoryQuota: Number, // Base + sum(active addon memory)
  usedVisitorMemory: Number,    // Count of active VisitorMemoryEntry records
  isMemoryFull: Boolean,        // Computed flag: usedVisitorMemory >= totalVisitorMemoryQuota
  // Billing cycle
  cycleStartDate: Date,         // When current billing cycle began
  cycleEndDate: Date,           // When current billing cycle ends (30 days later)
  lastResetAt: Date,            // When quota was last reset
  createdAt: Date,
  updatedAt: Date,
}

```

Key Behaviors:

- **Lazy Reset:** When a message arrives and `now > cycleEndDate`, automatically reset `usedMessages=0` and shift cycle by 30 days
- **Grace Period:** Allow 10 messages past hard limit before blocking chat
- **Memory Full Flag:** Computed on every message; triggers LRU eviction

Indexes: `tenantId` (unique), `cycleEndDate`

Model 4: VisitorMemoryEntry — Cross-session Customer Context

Purpose: Persistent customer memories (summarized conversations) used for personalization via RAG.

```

{
  _id: ObjectId,
  tenantId: ObjectId,           // FK to Tenant (indexed)
  visitorId: String,           // Unique ID per shop visitor (e.g., session UUID)
  sessionId: String,           // Which chat session this came from
  summary: String,             // Gemini-generated PII-scrubbed summary (2-4 sentences)
  embedding: Number[],         // 768-dim vector from text-embedding-004
  importance: Number,          // 1-10 (Gemini scores); used for LRU eviction
  lastAccessedAt: Date,        // Updated every time this memory is retrieved
  embeddedAt: Date | null,     // When vector was generated (null = failed)
  createdAt: Date,
  updatedAt: Date,
}

```

LRU Eviction: When `isMemoryFull=true`, find lowest-importance + oldest-accessed entry and delete it.

Indexes:

- { tenantId, visitorId }
- { tenantId, lastAccessedAt }
- { tenantId, importance, lastAccessedAt } — LRU eviction lookup

Model 5: ChatSession — Conversation History

Purpose: Stores all messages in a conversation for context injection to Gemini.

```

{
  _id: ObjectId,
  tenantId: ObjectId,           // FK to Tenant
  sessionId: String,           // Unique session ID (UUID), unique index
  visitorId: String | null,     // Anonymous initially, set after first message
  messages: [
    {
      role: "user" | "model",
      content: String,
      timestamp: Date,
    }
  ],
  messageCount: Number,        // Cached count
  sentiment: Number,           // Latest sentiment score (-10 to +10)
  handoffRequested: Boolean,    // Did customer or bot request human transfer?
  lastActiveAt: Date,          // When last message arrived
  createdAt: Date,
  updatedAt: Date,
}

```

TTL Index: Auto-delete after 30 days of inactivity

Model 6: VisitorProfile — CRM Tags & Metadata

Purpose: Merchant CRM view: tracks visitor behavior, auto-assigned tags, sentiment.

```
{
  _id: ObjectId,
  tenantId: ObjectId,
  visitorId: String,           // Unique per visitor
  tags: [
    "prospect",               // buying_intent detected
    "hot_lead",               // checkout_ready detected
    "price_shopper",          // price_inquiry detected
    "comparison",             // comparison_shopping detected
    "budget_sensitive",
    "repeat_visitor",         // 3+ sessions
    "handoff_requested",
    "vip",                    // manually set by merchant
  ],
  sessionCount: Number,       // # of chats this visitor had
  totalMessages: Number,      // Total messages sent
  sentimentAvg: Number,       // Rolling average sentiment
  lastSentiment: Number,      // Latest sentiment score
  handoffCount: Number,       // How many times handed off to human
  firstSeenAt: Date,
  lastSeenAt: Date,
  lastMessage: String,        // Last 200 chars of customer's message
  notes: String,              // Merchant's free-text notes (max 1000 chars)
  createdAt: Date,
  updatedAt: Date,
}
```

Unique Constraint: { tenantId, visitorId }

Auto-tagging Logic:

- "hot_lead" if latest buying_signal score > 0.8
- "price_shopper" if message contains price keywords
- "repeat_visitor" if sessionCount >= 3

Model 7: BotConfig — Merchant's Bot Personality Settings

Purpose: All merchant-configurable bot behavior (persona, guardrails, UI).

```

{
  _id: ObjectId,
  tenantId: ObjectId,          // Unique per tenant

  // Persona (what customers see)
  botName: String,             // Default: "Zudobot"
  botAvatar: String,           // Emoji or image URL
  backstory: String,           // "I'm a friendly sales assistant..."
  botIntro: String,            // "สวัสดีค่ะ! มีอะไรให้ช่วยไหม?"
  toneOfVoice: "FRIENDLY" | "PROFESSIONAL" | "PLAYFUL",
  primaryLanguage: "th" | "en" | "both",
  customKnowledge: String,     // Extra instructions (max 5000 chars)
  shippingPolicy: String,
  returnPolicy: String,

  // Sales guardrails
  maxDiscountPercent: Number,  // e.g., 10 (can't offer >10% discount)
  forbiddenTopics: String[],   // e.g., ["politics", "gambling"]
  handoffMessage: String,     // Custom message when handing off to human

  // UI
  quickReplies: String[],     // Max 5 quick reply buttons
  themeColor: String,         // Hex color code
  logoUrl: String,
  position: "bottom-right" | "bottom-left",
  autoOpenDelay: Number,      // ms to delay auto-open (0 = disabled)

  // Rate limiting
  maxMessagesPerSession: Number, // Max 20-200 messages per session

  // Operating hours
  operatingHours: {
    enabled: Boolean,
    timezone: String,          // "Asia/Bangkok"
    schedule: [
      { day: 0-6, open: "09:00", close: "21:00" }
    ],
    offlineMessage: String,    // Message when shop is closed
  },

  createdAt: Date,
  updatedAt: Date,
}

```

Model 8: KnowledgeBase — Product & Service Information

Purpose: Merchant's product/service catalog + policies, used for RAG semantic search.

```

{
  _id: ObjectId,
  tenantId: ObjectId,
  type: "text" | "url" | "pdf",
  title: String,
  content: String,                // Main text (max 20K chars)
  sourceUrl: String | null,      // For "url" type
  embedding: Number[],           // 768-dim vector (auto-generated)
  embeddingUpdatedAt: Date | null,
  isActive: Boolean,             // Can be used in RAG
  createdAt: Date,
  updatedAt: Date,
}

```

Example:

```

{
  title: "Summer Collection",
  content: "Our 2025 summer dresses are made from breathable cotton...",
  embedding: [0.123, -0.456, ...],
  type: "text",
  isActive: true
}

```

Model 9: Product — E-commerce Product Master Data

Purpose: Inventory & pricing for each product in merchant's store.

```

{
  _id: ObjectId,
  tenantId: ObjectId,
  name: String,                  // "Summer Dress Blue M"
  price: Number,                 // THB (-1 = "ติดต่อสอบถาม", 0 = free)
  priceSuffix: String,           // "/เดือน", "/ชิ้น"
  shortDescription: String,      // ~2000 chars
  slug: String,                  // URL-safe identifier
  stock: Number | null,          // null = unlimited
  variants: String[],            // ["Blue", "Red", "XL", "M"]
  isActive: Boolean,
  embedding: Number[],           // For semantic search
  embeddedAt: Date | null,
  createdAt: Date,
  updatedAt: Date,
}

```

Note: May be synced from external e-commerce platform (Shopify, custom API, etc.)

Model 10: TenantPurchase — Billing Invoice History

Purpose: Audit trail of all package purchases by a merchant.

```
{
  _id: ObjectId,
  tenantId: ObjectId,           // FK to Tenant
  packageSlug: String,         // Which package was bought
  packageName: String,         // Denormalized (e.g., "Starter Monthly")
  amount: Number,              // THB paid
  purchasedAt: Date,
  validFrom: Date,             // Effective date
  validTo: Date | null,        // Expiry date (null = no expiry)
  status: "active" | "expired" | "cancelled",
  note: String,                // Admin notes or reference number
  createdAt: Date,
  updatedAt: Date,
}
```

Indexes: { tenantId, purchasedAt }, { tenantId, status }

Model 11: CustomCommand — Merchant's Custom AI Instructions

Purpose: Merchant-specific AI behaviors injected in Layer 3 of system prompt.

```
{
  _id: ObjectId,
  tenantId: ObjectId,
  commandType: "SYSTEM_PROMPT_ADDON" | "AUTO_REPLY" | "SALES_STRATEGY",
  label: String,               // "Auto-reply to store hours question"
  triggerKeywords: String[],   // ["jam", "เปิด", "ปิด"] for AUTO_REPLY
  commandContent: String,      // The instruction text (max 3000 chars)
  priority: Number,           // 1-100 (higher = runs first)
  isActive: Boolean,
  validationWarning: String,   // If rules pre-scan found issues
  createdBy: String,
  updatedBy: String,
  createdAt: Date,
  updatedAt: Date,
}
```

Types:

- SYSTEM_PROMPT_ADDON : Extra system instructions (merged into prompt)
- AUTO_REPLY : If message matches keywords, send this reply
- SALES_STRATEGY : Coaching on how to upsell certain products

Model 12: KnowledgeGap — What Bot Can't Answer

Purpose: Track unanswered questions → inform merchant what to add to KB.

```
{
  _id: ObjectId,
  tenantId: ObjectId,
  sessionId: String,
  query: String,                // Customer's question (max 300 chars)
  frequency: Number,           // How many times this q was asked
  resolved: Boolean,           // Merchant added to KB or not
  createdAt: Date,
  updatedAt: Date,
}
```

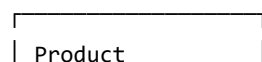
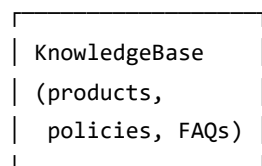
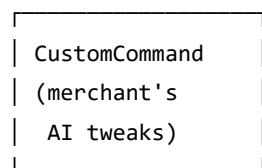
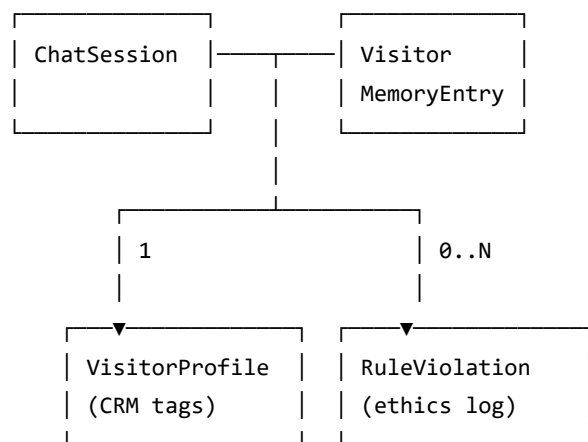
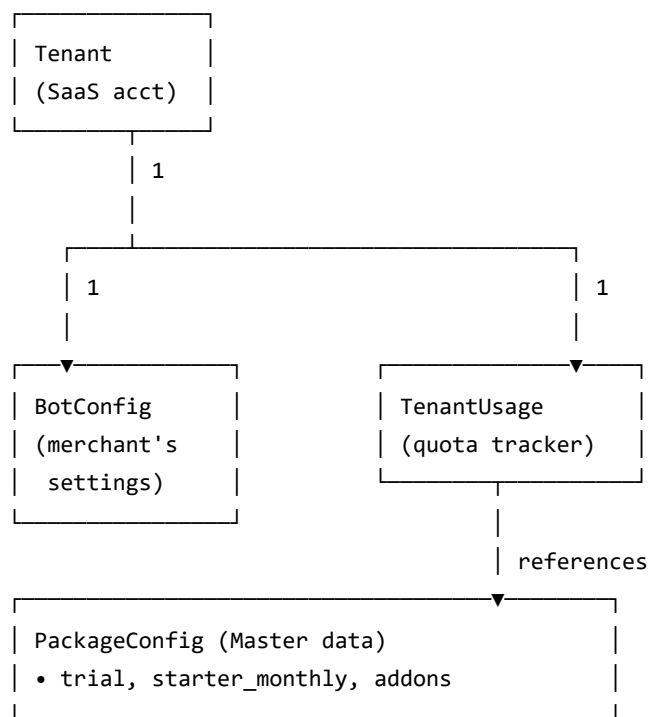
Unique Constraint: { tenantId, query } (deduplicate by tenant + normalized query)

Model 13: RuleViolation — Ethics Log

Purpose: Audit trail when bot violates constitutional rules.

```
{
  _id: ObjectId,
  tenantId: ObjectId | null,    // null if system-level violation
  sessionId: String,
  ruleIds: String[],           // e.g., ["A1", "C1"]
  category: String,            // "medical_claim", "dark_pattern", "pii_request"
  triggerText: String,         // What triggered it (max 300 chars)
  action: "allow" | "block" | "redirect_human" | "emergency",
  layer: "pre" | "post",       // Pre-generation or post-generation catch
  createdAt: Date,
}
```

6.2 Model Relationships (Entity-Relationship Diagram)



| (inventory, |
| pricing) |

| KnowledgeGap |
| (unanswered Qs) |

| TenantPurchase |
| (invoice history) |

6.3 Data Validation Rules

Model	Field	Validation
Tenant	plan	Enum: trial, starter, pro, enterprise
PackageConfig	price	>= 0; currency THB
TenantUsage	usedMessages	0 <= usedMessages <= totalMessageQuota + 10
VisitorMemoryEntry	importance	1–10 (integer)
BotConfig	maxDiscountPercent	0–100
BotConfig	maxMessagesPerSession	1–200
Product	price	-1 (ask), 0 (free), or > 0
Product	stock	null (unlimited) or >= 0
CustomCommand	commandContent	max 3000 chars; no